15

Lecture 15

Software-Defined Networking (SDN)

شبکه‌سازی نرم‌افزارمحور (SDN)

2G — 3G — 4G — **5G** — 6G

COURSE PART
Part III – Software-driven architectures (Lectures 15–18)

ESTIMATED STUDY TIME
6–8 hours (3 h reading, 3–5 h exercises)

DIFFICULTY
Intermediate

PREREQUISITES
IP routing and switching; Lecture 13 (CUPS); M/M/1 queue (Lecture 12)

In this lecture

1 Learning objectives . . . 2
2 Prerequisite knowledge . . . 2
3 Introduction and motivation . . . 3
4 Fundamental concepts . . . 3
5 Detailed technical explanation . . . 5
6 Mathematical models . . . 7
7 Network architecture and diagrams . . . 8
8 Step-by-step procedures . . . 8
9 Worked examples . . . 9
10 Technology comparison . . . 9
11 Key takeaways . . . 10
12 Exercises . . . 10
13 Further reading . . . 12

Original lecture notes for the course *Advanced Communication Networks*. The lecture sequence follows the course syllabus (26 lectures, 2G to 6G). Equations use identical notation in the English and Persian editions. Companion edition: Persian (`lecture-15-fa.pdf`).

1 Learning objectives

After studying this lecture you should be able to:

- **LO1** **Explain** the separation of control and data planes and the motivation for logically centralised control.
- **LO2** **Describe** the SDN layers and interfaces (northbound, southbound) and the OpenFlow switch model: flow tables, pipeline, groups and meters.
- **LO3** **Trace** reactive and proactive flow installation and **compute** their latency and table requirements.
- **LO4** **Explain** data-plane programmability with P4.
- **LO5** **Analyse** controller placement and controller load.
- **LO6** **Identify** SDN principles in mobile networks (CUPS, PFCP, transport and slicing).

2 Prerequisite knowledge

You should know how IP routers and Ethernet switches forward packets (longest-prefix match, MAC learning) and that routing protocols such as OSPF and BGP compute forwarding tables in a distributed way. From the course, you need CUPS and PFCP (Lecture 13) and the M/M/1 delay formula (Lecture 12).

Key terms. The vocabulary of this lecture with its Persian equivalents:

Term	Persian	Meaning in this lecture
Control plane	صفحهٔ کنترل	Logic that decides how traffic is forwarded.
Data plane	صفحهٔ داده	Per-packet forwarding according to installed state.
SDN controller	کنترل‌گر SDN	Logically centralised software that programs the data plane.
Southbound API	API جنوبی	Controller–device interface (OpenFlow, P4Runtime, NETCONF).
Northbound API	API شمالی	Controller–application interface (e.g. REST).
Flow table	جدول جریان	Prioritised match–action rules in a switch.
Packet-In / Flow-Mod	پیام‌های OpenFlow	Switch-to-controller packet report / rule installation.
Reactive / proactive	واکنشی / پیش‌کنشی	Rules installed on first packet / in advance.
P4	زبان P4	Language for programming protocol-independent packet processors.
TCAM	حافظهٔ آدرس‌پذیر محتوایی سه‌حالته	Hardware memory for wildcard matching.

3 Introduction and motivation

In a traditional network every router runs its own control software: it exchanges routing messages with its neighbours, computes paths and fills its forwarding table. Control and forwarding are bundled in each box, and each vendor implements the protocols in its own closed software. This design is robust, but it makes the network hard to change: a new forwarding behaviour needs a new protocol, a standard and years of implementation, and network-wide policies (traffic engineering, isolation, access control) must be expressed indirectly through many per-box configurations.

Software-defined networking (SDN) separates the two planes [5], [6]. Forwarding devices become simple, programmable match–action engines; a logically centralised controller with a global view computes their state and installs it over an open interface. Network behaviour becomes software that can be written, tested and deployed like other software. Mobile networks adopted the same idea: CUPS and PFCP (Lecture 13) are SDN applied to packet gateways, and the 5G core with its UPF is built on it. This lecture opens Part III, which covers the software technologies – SDN, NFV, service-based architecture – on which 5G networks rest.

4 Fundamental concepts

4.1 Three principles

1. **Separation of control and data planes**: devices forward; controllers decide.
2. **Logically centralised control**: one (possibly distributed and replicated) controller has a global view of topology and state.
3. **Programmability**: open interfaces let software applications define network behaviour.

4.2 Layers and interfaces

RFC 7426 describes SDN in terms of planes – forwarding, operational, control, management and application – and the abstraction layers between them [4]. In the common simplified view (Figure 1) applications use a *northbound* API (often REST) of the controller, and the controller programs devices through a *southbound* API: OpenFlow [7] for flow tables, P4Runtime for P4-programmed devices, or NETCONF/YANG for configuration.

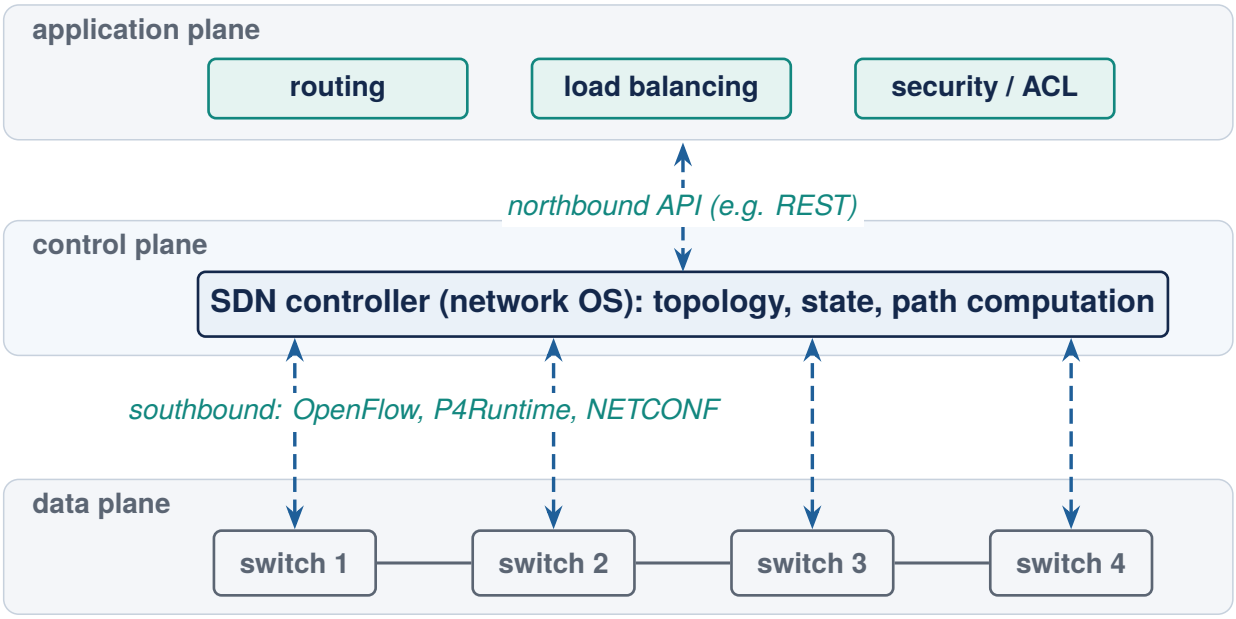


Figure 1. SDN layers: applications, controller and data plane with northbound and southbound APIs.

4.3 The OpenFlow switch

Definition 1 • Flow entry

A flow entry consists of match fields (ingress port and packet header fields from Ethernet to transport layer), a priority, counters, instructions (apply or write actions, go to another table), idle and hard timeouts, and an opaque cookie [7]. A packet is matched against the entries of a table; the highest-priority match wins.

A switch contains a pipeline of flow tables (Figure 2), a *group table* (for multicast, load balancing with *select* groups and fast failover) and a *meter table* (rate limiting). If no entry matches (*table miss*), the table-miss entry decides: drop, continue, or send the packet to the controller. The switch talks to the controller over a TCP connection (port 6653, optionally TLS).

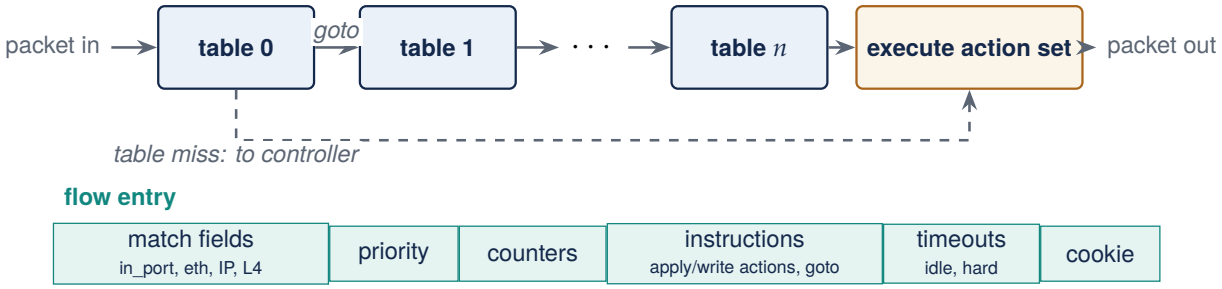


Figure 2. OpenFlow pipeline with multiple flow tables, and the fields of a flow entry.

5 Detailed technical explanation

5.1 OpenFlow messages

- **Controller-to-switch**: Features (capabilities), Flow-Mod (add, modify, delete entries), Group-Mod, Meter-Mod, Packet-Out (send a packet from a port), Multipart/Stats (read counters), Barrier (ensure earlier messages are processed).
- **Asynchronous**: Packet-In (a packet sent to the controller, by default with its first 128 bytes and a buffer identifier), Flow-Removed (an entry expired), Port-Status (a port changed).
- **Symmetric**: Hello, Echo (liveness), Error.

5.2 Reactive and proactive control

In *reactive* mode the first packet of a new flow misses, the switch sends a Packet-In, and the controller computes a path and installs entries along it before releasing the packet with Packet-Out (Figure 3). Fine-grained control is possible, but every new flow pays a controller round trip, and the controller load grows with the flow arrival rate. In *proactive* mode the controller installs (usually wildcard) rules in advance, from its knowledge of topology and policy; no packet waits, the controller is off the critical path, but rules must be aggregated to fit the tables. Practical deployments mix both: proactive rules for the bulk of traffic, reactive handling for exceptions.

5.3 Data-plane programmability: P4

OpenFlow fixes the header fields a switch can match; adding a new protocol requires a new version of the specification and of the hardware. P4 [3] goes one step further: the program defines the headers, the parser state machine, the match–action tables and the control flow, and a compiler maps it to a programmable target (switch ASIC, smart NIC, software switch) organised as a parser, ingress and egress match–action pipelines and a deparser (Figure 4). The controller then fills the tables at run time through P4Runtime. P4 has been used, for example, to implement GTP-U encapsulation and UPF functions directly in switch hardware.

5.4 Distributed controllers and consistency

A single controller is a single point of failure and a bottleneck. Production controllers run as clusters that replicate state with consensus protocols and partition switches among instances; a switch connects to several controllers with master/slave (later primary/secondary) roles. Updat-

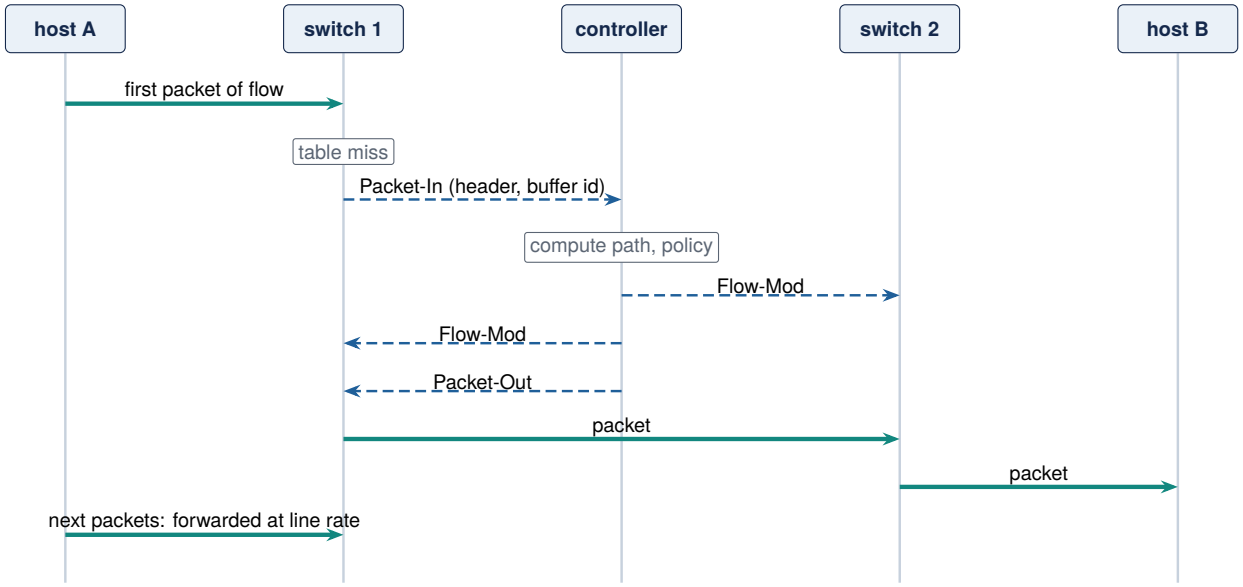


Figure 3. Reactive flow set-up: table miss, Packet-In, Flow-Mod along the path, Packet-Out.

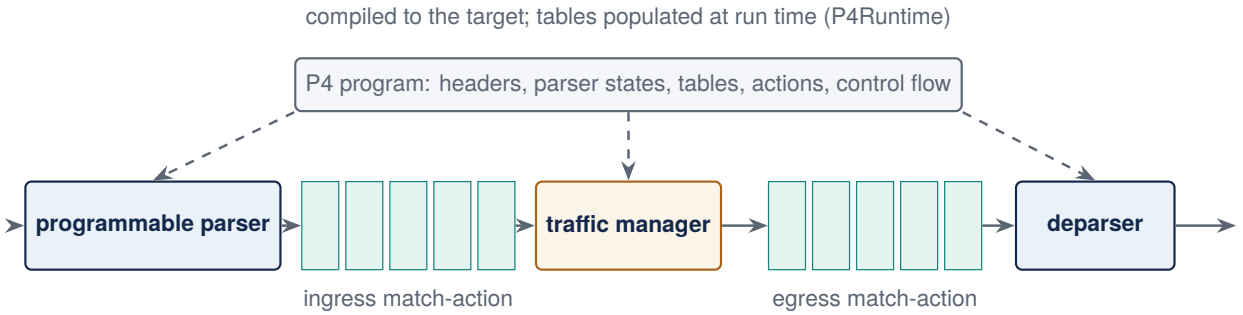


Figure 4. Protocol-independent switch architecture programmed with P4.

ing many switches is not atomic: during an update some packets may see old rules on one switch and new ones on the next, causing loops or policy violations. *Consistent update* techniques tag packets with a configuration version at the ingress and install the new version on all switches before flipping the tags, so that each packet sees either the old or the new configuration, never a mix.

5.5 SDN in mobile networks

- **Gateways**: CUPS (Lecture 13) separates SGW/PGW control from user-plane functions; PFCP plays the role of a southbound protocol with rules (PDR, FAR, QER, URR) that resemble OpenFlow entries.
- **5G core**: the SMF controls UPFs over N4 with PFCP (Lecture 23).
- **Transport**: SDN controllers configure backhaul and fronthaul paths, segment routing and transport slices.
- **RAN**: the O-RAN architecture opens RAN control to software applications through a RAN intelligent controller – the SDN idea applied to radio resource management.

6 Mathematical models

6.1 Reactive flow-set-up latency

Model. The first packet of a flow waits for the switch–controller round trip τ and the controller's processing. With S switches each generating λ_s new flows per second and a controller modelled as an M/M/1 server of rate μ,

$$T_{\text{setup}} \approx \tau + \frac{1}{\mu - S\lambda_s}, \qquad S\lambda_s < \mu. \tag{1}$$

Example. $\tau = 2\,\text{ms}$, $S = 100$, $\lambda_s = 1000\,\text{s}^{-1}$, $\mu = 1.5 \times 10^5\,\text{s}^{-1}$: $T_{\text{setup}} = 2 + 0.02 = 2.02\,\text{ms}$. At $S\lambda_s = 1.49 \times 10^5$ the queueing term rises to 1 ms (Figure 5).

Interpretation. For moderate load the propagation delay to the controller dominates, which argues for controllers close to the switches; near saturation the queueing term explodes and Packet-In storms (e.g. during a scan or a failure) can overload the controller – one reason to prefer proactive rules.

6.2 Flow-table occupancy

Model. With exact-match (per-flow) rules, new flows arrive at rate λ and each entry stays for the flow duration plus the idle timeout, $\bar{T}$ on average. By Little's law the mean number of entries is

$$\bar{N} = \lambda\,\bar{T}. \tag{2}$$

Example. $\lambda = 2000\,\text{s}^{-1}$, mean flow duration 10 s, idle timeout 10 s: $\bar{N} = 40\,000$ entries – far more than the few thousand TCAM entries of many switches. Per-flow reactive control does not scale to such switches; wildcard rules, aggregation, or larger exact-match tables (hash-based) are needed.

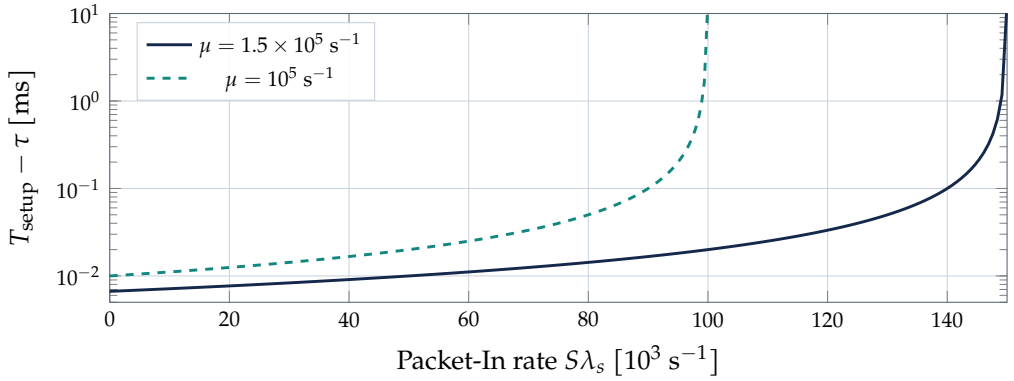


Figure 5. Controller queueing delay versus Packet-In rate (Eq. (1)).

6.3 Controller placement

Model. For switches $v \in V$ and controller locations C, with $d(v, c)$ the latency between them, the average and worst-case switch–controller latencies are

$$\bar{L}(C) = \frac{1}{|V|} \sum_{v \in V} \min_{c \in C} d(v, c), \qquad L_{\max}(C) = \max_{v \in V} \min_{c \in C} d(v, c). \tag{3}$$

Minimising $\bar{L}$ for $|C| = k$ is the k-median problem; minimising $L_{\max}$ is the k-centre problem.

Example. Five switches on a line, 1 ms apart: one controller at the centre gives $\bar{L} = 1.2$ ms ($L_{\max} = 2$ ms); at an end, $\bar{L} = 2$ ms. Two controllers at the second and fourth switches give $\bar{L} = 0.6$ ms and $L_{\max} = 1$ ms.

7 Network architecture and diagrams

In a data centre, the SDN controller programs the leaf–spine fabric and the virtual switches in the servers; in a WAN, it computes traffic-engineered paths between sites; in a mobile operator, SDN controllers manage transport networks and, through the core's control functions, the user-plane nodes. The controller is itself a distributed application deployed in redundant instances, often on the virtualised infrastructure of Lecture 16.

8 Step-by-step procedures

Procedure 1 • Reactive flow installation

1. The first packet of a new flow arrives at switch 1 and misses all entries; the table-miss entry sends it to the controller as Packet-In (header and buffer identifier).
2. The controller identifies source and destination, checks policy and computes a path with its topology view.
3. It sends Flow-Mod messages to all switches on the path (destination side first, to avoid further misses), optionally followed by a Barrier.

4. It sends Packet-Out to switch 1 to release the buffered packet along the new path.
5. Subsequent packets match the new entries and are forwarded at line rate; entries expire after the idle timeout and Flow-Removed reports their counters.

Procedure 2 • Topology discovery

1. On connection, each switch reports its ports (Features reply).
2. The controller sends, via Packet-Out, a link-discovery frame (LLDP) on every port of every switch.
3. A neighbouring switch receives the frame and, by a pre-installed rule, returns it as Packet-In with its ingress port.
4. From (sending switch, port) and (receiving switch, port) the controller learns each link; it repeats periodically and updates the topology on Port-Status events.

9 Worked examples

Worked example 1 • Set-up delay budget

Problem. A campus network has 100 switches; each sees 1000 new flows per second. The controller handles 1.5×10^5 Packet-In/s and is 1 ms away (one way). What delay does the first packet of a flow see, and what fraction of short web flows (10 packets, 20 ms) is it?
Solution. $\tau = 2$ ms, queueing 0.02 ms (Eq. (1)): about 2 ms, i.e. 10 % of the flow's duration. The controller runs at 67 % load; a 50 % increase in flow rate would saturate it.

Worked example 2 • Table size

Problem. A top-of-rack switch with 4000 TCAM entries sees 2000 new flows/s, each lasting 10 s on average, idle timeout 10 s. Can reactive per-flow rules be used? What if the idle timeout is 1 s?
Solution. $\bar{N} = 2000 \times 20 = 40\,000 > 4000$: no. With 1 s idle timeout, $\bar{N} = 22\,000$: still no. Rules must be aggregated, e.g. per destination prefix (proactive), which needs entries per destination rather than per flow.

Worked example 3 • Placement

Problem. Verify the values of the line example in Section 6.3.
Solution. Centre (switch 3): distances 2, 1, 0, 1, 2; mean 1.2 ms, max 2 ms. End (switch 1): 0, 1, 2, 3, 4; mean 2 ms, max 4 ms. Controllers at 2 and 4: 1, 0, 1, 0, 1; mean 0.6 ms, max 1 ms.

10 Technology comparison

Not a replacement for routing. SDN does not abolish distributed protocols: many deployments keep BGP or IS-IS for basic reachability and fast local failover, and use the controller for policy

Table 1. Traditional distributed control versus SDN.

	Distributed control (OSPF, BGP)	SDN
Control location	In every device	Logically centralised controller
View	Local, built by protocol exchange	Global topology and state
New behaviour	New protocol, standard, firmware	New controller application
Failure handling	Local, fast, autonomous	Needs controller or pre-installed backups
Scalability limit	Protocol convergence, table size	Controller capacity, control-channel latency
Policy expression	Indirect (metrics, filters per box)	Direct, network-wide
Vendor coupling	Hardware and software bundled	Open interfaces, merchant silicon

and optimisation on top (hybrid SDN).

11 Key takeaways

Key takeaways

- SDN separates control from forwarding, centralises control logically and makes the network programmable.
- OpenFlow switches: prioritised flow tables in a pipeline, groups, meters; Packet-In, Flow-Mod, Packet-Out.
- Reactive control costs $T_{\text{setup}} \approx \tau + 1/(\mu - S\lambda_s)$ per new flow; proactive control needs aggregated rules.
- Table occupancy follows Little's law, $\bar{N} = \lambda \bar{T}$; per-flow rules rarely fit TCAMs.
- P4 makes the parser and pipeline programmable; controller placement is a k-median/k-centre problem.
- CUPS/PFCP and the 5G UPF apply SDN to the mobile core.

12 Exercises

Exercise 1 • Level 1 – conceptual • Which plane?

Classify as control or data plane: (a) OSPF shortest-path computation; (b) TTL decrement; (c) installing a Flow-Mod; (d) matching a packet against a TCAM; (e) LLDP-based topology discovery.

Exercise 2 • Level 1 – conceptual • Reactive or proactive?

For each case choose reactive or proactive control and justify: (a) a data-centre fabric with destination-based forwarding; (b) a campus network where guests must authenticate before their first flow is allowed; (c) a mobile backhaul network.

Exercise 3 • Level 2 – numerical • Control-channel bandwidth

A controller receives 10^5 Packet-In messages per second, each carrying 128 bytes of packet plus 32 bytes of OpenFlow and TCP/IP overhead (amortised). Compute the control-channel bandwidth. How does it change if the switch sends whole 1500-byte packets instead of buffering them?

Exercise 4 • Level 2 – numerical • Timeouts

Using Eq. (2), find the idle timeout that keeps the mean occupancy at 15 000 entries for 1000 new flows/s with mean duration 8 s. What is the price of a very short idle timeout?

Exercise 5 • Level 3 – analytical • Placement on a ring

Six switches form a ring with 1 ms links. Compute $\bar{L}$ and $L_{\max}$ for one controller, and for two controllers placed optimally. What happens to the worst case if one of the two controllers fails?

Exercise 6 • Level 3 – analytical • Consistent update

A path change moves traffic from route A–B–D to A–C–D. Show how updating switch A before installing rules on C can drop packets, and describe a version-tagging update that avoids both drops and mixing. How many rules does it need temporarily?

Solutions

Solution to Exercise 1

(a) control; (b) data; (c) control (the act of installing; the entry is then used by the data plane); (d) data; (e) control.

Solution to Exercise 2

(a) Proactive: destinations are known, traffic volume is huge, and per-flow set-up delay is unacceptable. (b) Reactive for the first flow of each guest (the controller must see it to apply authentication), then proactive rules for authorised hosts. (c) Proactive: paths are planned and must survive controller outages.

Solution to Exercise 3

$10^5 \times 160\,\text{bytes} = 16\,\text{MB/s} = 128\,\text{Mbit/s}$. With whole packets: $10^5 \times 1532 \times 8 = 1.23\,\text{Gbit/s}$ – ten times more, and the controller must also store or forward them. Buffering in the switch saves control-channel capacity.

Solution to Exercise 4

$15\,000 = 1000\,(8 + T_{\text{idle}}) \Rightarrow T_{\text{idle}} = 7\,\text{s}$. A very short idle timeout evicts entries of flows that pause briefly; their next packet misses again and causes another Packet-In, increasing controller load and latency.

Solution to Exercise

One controller: distances 0, 1, 1, 2, 2, 3; $\bar{L} = 9/6 = 1.5\,\text{ms}$, $L_{\max} = 3\,\text{ms}$. Two opposite controllers: 0, 1, 1, 0, 1, 1; $\bar{L} = 4/6 = 0.67\,\text{ms}$, $L_{\max} = 1\,\text{ms}$. If one fails, the survivor serves all with $L_{\max} = 3\,\text{ms}$ – the placement must be evaluated for failure cases too.

Solution to Exercise

If A forwards to C before C has a rule for the flow, C misses and drops (or sends to the controller). Order matters: install on C first, then switch A ("make before break"). For general updates: install the new rules on C and D matching version tag 2 (while version-1 rules remain), then make A stamp tag 2 on new packets; once old packets have drained, delete version-1 rules. Temporarily the affected switches hold both versions – up to twice the rules.

13 Further reading

McKeown et al. [6] introduced OpenFlow, whose current switch specification is published by the ONF [7]; Kreutz et al. [5] survey SDN comprehensively; RFC 7426 [4] defines SDN layers and terminology; Bosshart et al. [3] present P4. For the mobile context see TS 23.214 [1] and TS 29.244 [2].

References

[1] 3GPP, *Architecture enhancements for control and user plane separation of EPC nodes*, TS 23.214, 3rd Generation Partnership Project. [Online]. Available: https://www.3gpp.org/DynaReport/23214.htm.

[2] 3GPP, *Interface between the control plane and the user plane nodes*, TS 29.244, 3rd Generation Partnership Project. [Online]. Available: https://www.3gpp.org/DynaReport/29244.htm.

[3] P. Bosshart, D. Daly, G. Gibb, *et al.*, "P4: Programming protocol-independent packet processors", *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[4] E. Haleplidis, K. Pentikousis, S. Denazis, J. Hadi Salim, D. Meyer, and O. Koufopavlou, "Software-defined networking (SDN): Layers and architecture terminology", IRTF, RFC 7426, 2015. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7426.

[5] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey", *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[6] N. McKeown, T. Anderson, H. Balakrishnan, *et al.*, "OpenFlow: Enabling innovation in campus networks", *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[7] Open Networking Foundation, *OpenFlow switch specification, version 1.5.1*, Open Networking Foundation, 2015.